



## Contenido

|                                                             |   |
|-------------------------------------------------------------|---|
| 1. ¿ QUÉ ES WPF Y POR QUÉ LO VAMOS A USAR EN EMPRESA? ..... | 1 |
| 2. CARACTERÍSTICAS DE WPF .....                             | 2 |
| 2.1. Interfaz gráfica declarativa .....                     | 2 |
| 2.2. Binding .....                                          | 2 |
| 2.3. Diseño dinámico .....                                  | 2 |
| 2.4. Plantillas .....                                       | 2 |
| 2.5. Estilos.....                                           | 2 |
| 2.6. DirectX vs GDI .....                                   | 3 |
| 3. CONTROLES.....                                           | 3 |
| 4. DISEÑO .....                                             | 3 |
| 5. WPF WINDOW, CONFIGURACIÓN DE LA VENTANA PRINCIPAL.....   | 3 |
| 5.1. CLASS.....                                             | 4 |
| 5.2. REFERENCIAS.....                                       | 4 |

## 1. ¿ QUÉ ES WPF Y POR QUÉ LO VAMOS A USAR EN EMPRESA?

Windows Presentation Foundation (**WPF**) es la tecnología **actual** de Microsoft para crear interfaces de escritorio modernas en el ecosistema .NET (incluida la versión .NET 8/9).

Es el **sucesor directo** de Windows Forms y aporta mejoras radicales que cualquier empresa demanda hoy:

- Interfaz **declarativa** con **XAML** (separación total entre diseño y lógica → mantenimiento más fácil).
- **Binding** potente y automático entre datos y interfaz.
- Diseño **vectorial** y **dinámico** que se adapta perfectamente a cualquier resolución y DPI (retina, 4K, ultrawide...).



- **Estilos y plantillas** reutilizables (igual que CSS pero mucho más potentes).
- Renderizado con **DirectX** → animaciones fluidas, efectos visuales y multimedia de alto rendimiento.
- Soporte completo de **accesibilidad** e **internacionalización** desde el primer momento.

**Con WPF creamos interfaces profesionales, mantenibles y atractivas** que cumplen los estándares de usabilidad y experiencia de usuario exigidos en el mercado laboral actual.

**Los controles se pueden modificar de tres formas complementarias** (las tres las dominaremos en esta unidad):

- Vista Diseño (Visual Studio).
- Código XAML (declarativo).
- Code-behind en C# (lógica).

## 2. CARACTERÍSTICAS DE WPF

### 2.1. Interfaz gráfica declarativa

WPF permite crear interfaces gráficas empleando XAML (es un lenguaje de marcas, etiquetas), muy similar a HTML.

En Visual Studio tenemos la vista diseño similar al de las aplicaciones Windows Forms. La diferencia es que hay un editor donde puedes escribir el código XAML directamente.

### 2.2. Binding

Es un mecanismo para enlazar los elementos de la interfaz de usuario con los objetos que contienen la información a mostrar.

El caso más típico de data binding es el enlazar un control de la interfaz de usuario con un valor o registro de una base de datos. Con **Windows Forms** se utiliza el **databinding**.

### 2.3. Diseño dinámico

La organización de los componentes de una aplicación puede ser muy complicado. Además, es necesario considerar las diferentes resoluciones y características de una pantalla de un usuario.

En este sentido, las interfaces WPF generadas son **vectoriales**, es decir, se podrán escalar a diferentes tamaños sin perder definición, dependiendo de cómo se haya definido el diseño.

### 2.4. Plantillas

Se pueden crear elementos reutilizables para la interfaz gráfica. Existen dos tipos de plantillas en WPF: plantillas de control y plantillas de datos.

Las **plantillas de control** (control templates) redefinen el aspecto visual de un control, permitiendo, por ejemplo, cambiar la apariencia de todos los botones de una aplicación desde un único punto.

Las **plantillas de datos** (data templates) definen la manera en que los datos son mostrados, asocian una plantilla a una entidad o clase de la aplicación. Es decir, siempre que se muestre ese tipo de dato se utilizará la misma plantilla.

### 2.5. Estilos

Los estilos en WPF son muy parecidos a las hojas de estilos en cascada, CSS, para HTML. Incluso más potentes, permitiendo manejar cualquier característica gráfica que te imagines, márgenes, espacios, colores, dimensiones y muchos más.

## 2.6. DirectX vs GDI

Mientras que su predecesor está basado en la interfaz de diseño de gráficos GDI (Graphics Design Interface), WPF se basa en DirectX. Esta API está enfocada al desarrollo de multimedia y particularmente a la programación de juegos.

## 3. CONTROLES

Un control es un término genérico que se aplica a una categoría de clases de WPF dentro de la ventana o página, tienen una apariencia e implementan un comportamiento determinado.

Algunos de los más empleados según su **funcionalidad**:

- **Botones:** Button.
- **Presentación de datos:** DataGrid, ListView y TreeView.
- **Presentación y selección de fechas:** Calendar y DatePicker.
- **Entrada:** TextBox, RichTextBox y PasswordBox.
- **Diseño:** Border, Canvas, DockPanel, Grid, GridSplitter, GroupBox, ScrollBar, StackPanel.
- **Multimedia:** Image, MediaElement.
- **Menús:** Menu y ToolBar.
- **Navegación:** Frame, TabControl.
- **Selección:** CheckBox, ComboBox, ListBox, RadioButton y Slider.  **Información del usuario:** Label, ProgressBar, StatusBar.

## 4. DISEÑO

Al crear una interfaz de usuario, se organizan los **controles** según su ubicación y tamaño. Un requisito fundamental de cualquier diseño es adaptarse a los cambios de tamaño de la ventana y de configuración de pantalla.

WPF incluye varios controles de diseño:

- **Canvas:** los controles secundarios se colocan explícitamente mediante coordenadas relativas al área de Canvas.
- **DockPanel:** los controles secundarios se organizan de forma horizontal o vertical.
- **Grid:** los controles secundarios se colocan por filas y columnas.
- **StackPanel:** los controles secundarios se apilan vertical u horizontalmente.
- **WrapPanel:** los controles secundarios se sitúan en orden de izquierda a derecha y se ajustan a la línea siguiente cuando hay más controles de los que caben en la línea actual.

## 5. WPF WINDOW, CONFIGURACIÓN DE LA VENTANA PRINCIPAL

El control de **WPF Window**, será el primer control que debemos configurar para adaptarlo a nuestro proyecto.

Cuando creamos un nuevo proyecto, de forma automática Visual Studio nos añade lo siguiente:

```
<Window x:Class="Primera_Interfaz.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:Primera_Interfaz"
        mc:Ignorable="d"
        Title="Primera Interfaz" Height="450" Width="800" Background="#FFE2E2DF">
```

## 2.7. CLASS

Un WPF Window está compuesto por dos archivos:

- Un archivo XAML (.xaml) .
- Un archivo de código (.cs) (code behind).

Para que ambas estén relacionadas, es necesario indicar en el XAML que clase utilizar. En el archivo .cs le daremos funcionalidad.

## 2.8. REFERENCIAS

Además de la clase, hay definiciones de espacio de nombres que son referencias a otras librerías necesarias para la aplicación.

### A) TITLE

Esta propiedad establece u obtiene el título de la ventana.

### B) WIDTH Y HEIGHT

Estas propiedades establecen u obtienen el ancho y el alto de la ventana. Indican el tamaño con el que se abrirá la aplicación.

### C) MINWIDTH, MINHEIGHT, MAXWIDTH Y MAXHEIGHT

Permitir acotar el tamaño de la ventana por arriba y por abajo.

### D) RESIZEMODE

Establecer cómo podemos cambiar el tamaño del WPF Window. Puede tener cuatro valores posibles que determinarán su comportamiento.

- **NoResize:** no permite cambiar el tamaño de la venta. Esta siempre se mantendrá fija.
- **CanMinimize:** solo permite minimizar la ventana y maximizar desde la barra de tareas. El botón maximizar se muestra pero está deshabilitado.

- **CanResize:** esta es la opción por defecto. Permite modificar de manera libre el tamaño de la ventana.
- **CanResizeWithGrip:** esta opción es igual que la anterior. La diferencia es que nos muestra un control de ajuste de tamaño en la esquina inferior derecha.

#### E) SIZETOCONTENT

Indica cómo se adapta la ventana al contenido dentro de ella. Puede tener cuatro valores posibles:

- **Manual:** es el valor por defecto. La ventana no cambia de tamaño según su contenido.
- **Width:** la ventana cambia de tamaño según el contenido solo en ancho.
- **Height:** la ventana cambia de tamaño según el contenido solo en alto.
- **WidthAndHeight:** la ventana cambia de tamaño según el contenido tanto en alto como en ancho.

#### F) WINDOWSTATE

Permite indicar el estado de la ventana cuando arranca la aplicación. Tiene 3 valores posibles.

- **Normal:** posición normal. Dependerá de la configuración del tamaño de la ventana.
- **Maximized:** ventana maximizada.  **Minimized:** ventana minimizada.